

Grafika 3D w bibliotece WebGL/GLSL

Laboratorium 12

1. Wprowadzenie

Celem laboratorium jest budowanie scen 3D z prostych prymitywów geometrycznych. Korzystamy z biblioteki Three.js, która udostępnia gotowe geometrie (sześcián, walec, stożek), a my składamy z nich scenę poprzez transformacje (translacja, rotacja, skala) i hierarchię obiektów. Podstawy WebGL i GLSL zostały omówione na poprzednich zajęciach.

2. Przestrzeń 3D i układ współrzędnych

W Three.js obowiązuje prawoskrętny układ współrzędnych: oś X w prawo, Y do góry, Z do obserwatora. Każdy obiekt na scenie ma własną pozycję, rotację i skalę względem swojego rodzica.

Kąty rotacji wyrażone są w radianach: pełny obrót to $2 * \text{Math.PI}$, kąt prosty to $\text{Math.PI} / 2$.

3. Prymitywy geometryczne

- `BoxGeometry(w, h, d)` — prostopadłościan
- `CylinderGeometry(rTop, rBottom, h, segments)` — walec
- `ConeGeometry(r, h, segments)` — stożek
- `SphereGeometry(r, wSeg, hSeg)` — sfera
- `TorusGeometry(r, tube)` — torus
- `TeapotGeometry(size)` — czajnik Utah (import z `three/addons/geometries/TeapotGeometry.js`)

Wszystkie nieprzekształcone prymitywy są wycentrowane w punkcie (0, 0, 0). Walec i stożek są zorientowane wzdłuż osi Y — wierzchołek stożka znajduje się na górze, a podstawa na dole.

4. Transformacje obiektów

Każdy obiekt typu `Mesh` ma trzy własności określające jego umiejscowienie w przestrzeni:

```
const mesh = new THREE.Mesh(geometry, material);

mesh.position.set(2, 0, -1);           // przesunięcie
mesh.rotation.set(0, Math.PI/4, 0);    // obrót (radiany)
mesh.scale.set(1, 3, 1);               // skala

scene.add(mesh);
```

Three.js automatycznie buduje z nich macierz świata stosowaną w vertex shaderze. Kolejność operacji jest stała: najpierw skala, potem rotacja, potem translacja.

5. Hierarchia obiektów — THREE.Group

Aby kilka obiektów zachowywało się jak jedna całość (poruszało, obracało, skalowało się razem), wkładamy je do wspólnej grupy:

```
const rotor = new THREE.Group();
rotor.add(blade1);
rotor.add(blade2);
rotor.add(blade3);
rotor.position.set(0, 5, 0);
scene.add(rotor);

renderer.setAnimationLoop(() => {
  rotor.rotation.z += 0.02;
  renderer.render(scene, camera);
});
```

Obiekty wewnątrz grupy mają pozycję względem grupy, nie względem świata. Mechanizm ten odpowiada dawnemu `glPushMatrix()` / `glPopMatrix()` w starym OpenGL i pozwala budować złożone obiekty z prostszych części (np. wirnik składa się z łopatek, ramię z segmentów, pojazd z koła + nadwozia).

Uwaga. Grupy można zagnieżdżać dowolnie głęboko. Łopatką wiatraka może być grupą zawierającą dwa stożki, a wirnik grupą zawierającą trzy takie łopatki. Obrót wirnika obróci wszystkie łopatki, a obrót pojedynczej łopatki obróci jej dwa stożki.

6. Animacja

Animacja w Three.js polega na modyfikowaniu własności obiektów (zwykle `position` lub `rotation`) w pętli renderowania:

```
renderer.setAnimationLoop(() => {
  rotor.rotation.z += 0.02;
  renderer.render(scene, camera);
});
```

Pętla wywoływana jest 60 razy na sekundę (zsynchronizowana z odświeżaniem ekranu). Inkrementacja 0.02 rad/klatkę daje około 200°/s, czyli pełen obrót co ~1.8 sekundy.

Literatura

- Three.js — dokumentacja oficjalna: <https://threejs.org/docs/>
- Three.js Manual: <https://threejs.org/manual/>
- D. Eck, *Introduction to Computer Graphics*, rozdział 7: <https://math.hws.edu/graphicsbook/c7/index.html>

7. Zadanie

Plik `lab12.html` pokazuje mały sześcian, który można obrócić, przeciągając myszą na płótnie. Zadaniem jest zastąpienie sześcianu dużym wiatrakem stojącym na prostokątnej podstawie. Łopatki wiatraka powinny

obracać się po włączeniu animacji. Każda łopatką ma być zbudowana z dwóch stożków. Dodanie czajnika stojącego na podstawie jest konieczne dla uzyskania oceny 5.